

FERRAMENTA DE VERSIONAMENTO GIT II

CARACTERÍSTICAS

Distribuído Git é um sistema de controle de versão distribuído. Cada cópia de um repositório Git contém todo o histórico de alterações, tornando-o altamente resiliente a falhas e permitindo o trabalho offline.

Pode-se iniciar um repositório Git local utilizando o comando `git init`.

RASTREAMENTO DE ALTERAÇÕES

Git rastreia todas as mudanças feitas em arquivos e diretórios permitindo que os desenvolvedores vejam quem fez quais alterações e quando.

Branching e merging, ramificação e mesclagem. É possível ter várias linhas-base.

O Git facilita a criação de ramificações (branches) independentes do código para desenvolvimento paralelo.

Posteriormente, a fusão (merge) dessas ramificações de volta ao ramo principal (branch master ou main).

HISTÓRICO DE REVISÕES

É uma linha do tempo completa e rastreável de todas as mudanças feitas em um projeto permitindo a recuperação de versões anteriores do código. Com as tags é possível reconstruir uma versão do software, com todos os seus componentes.

VELOCIDADE E DESEMPENHO

O Git é conhecido por sua velocidade e eficiência no gerenciamento de grandes repositórios de código-fonte.

GERENCIAMENTO DE CONFLITOS

Quando ocorrem conflitos entre diferentes versões de um arquivo o Git ajuda a resolvê-los e fundir as alterações de maneira controlada.

Os conflitos ocorrem quando múltiplas alterações são feitas em um mesmo arquivo por diferentes desenvolvedores. Nessas situações, o sistema auxilia na resolução desses conflitos, permitindo que se selecione a versão de código mais apropriada para ser integrada ao projeto.

No caso de conflitos, o Git permite decidir qual versão do código deve prevalecer, garantindo que a última modificação não sobreponha acidentalmente outras melhorias ou funcionalidades que já tenham sido implementadas.



FLEXIBILIDADE

O Git é altamente flexível e pode ser usado em uma variedade de fluxos de trabalho desde projetos individuais até projetos com grandes equipes de desenvolvimento.

O sistema foi pensado exatamente para permitir grandes frentes de trabalho, aproveitando o uso da web para facilitar a atuação de equipes remotamente. Um exemplo é um aplicativo desenvolvido durante um mestrado, no qual, mesmo com a equipe morando em diferentes localidades, foi possível fazer o gerenciamento e configuração adequados, colocando o software em produção. Essa ferramenta é ideal para ambientes com muitos desenvolvedores, inclusive aqueles que trabalham remotamente, pois facilita a colaboração.

COLABORAÇÃO

O Git é frequentemente usado em conjunto com plataformas de hospedagem de código como GitHub, GitLab e Bitbucket, facilitando a colaboração entre desenvolvedores de problemas (issues) CI/CD.

É importante lembrar que essas plataformas complementares são úteis para o processo de CI/CD, com o GitLab, por exemplo, sendo uma ferramenta de uso frequente em projetos de pesquisa e desenvolvimento.

RASTREAMENTO DE SUBMÓDULOS

O Git permite a inclusão de repositórios de código-fonte externos como submódulos tornando-o útil para projetos que dependem de bibliotecas ou componentes de terceiros.

No desenvolvimento, é comum incorporar funcionalidades de repositórios externos como submódulos no projeto, especialmente em linguagens como Java, que utiliza bibliotecas encapsuladas.

CARACTERÍSTICAS

CÓDIGO ABERTO

Git é um software de código aberto. Comunidade ativa e tem uma comunidade ativa de desenvolvedores que contribuem para seu aprimoramento e solução de problemas.

SEGURANÇA

O Git possui recursos de autenticação e autorização para proteger o acesso aos repositórios e garantir a integridade dos dados.

VERSIONAMENTO DE QUALQUER TIPO DE ARQUIVO

Embora tenha sido projetado para gerenciar código-fonte o Git pode ser usado para controlar versões de qualquer tipo de arquivo, incluindo documentos, imagens e dados.

O executável armazena apenas o binário correspondente, sem a capacidade de abrir e verificar alterações. Realiza, portanto, apenas um arquivamento, enquanto o código fonte possibilita o controle de versão.



10m

COMANDO → DESCRIÇÃO

`git init` → Inicializa um novo repositório Git no diretório atual.

`git clone <url>` → Clona um repositório remoto para o seu computador.

`git add <arquivo>` → Adiciona um arquivo específico para o índice (staging area).

`git add.` → Adiciona todos os arquivos modificados no diretório atual ao índice. É fundamental executar o “`git add`” antes do “`commit`”, para registrar as atualizações.

`git commit -m “<msg>”` → Cria um commit com uma mensagem descritiva.

O trabalho geralmente ocorre em um repositório local, sincronizado com um repositório central.

`git status` → Mostra o status atual do repositório (quais arquivos foram modificados, adicionados ou não).

`git log` → Exibe o histórico de commits do repositório.

`git diff` → Mostra as diferenças entre os arquivos modificados e o último commit.

`git branch` → Lista todas as branches locais ou cria uma nova se especificado (ex.: `git branch <nome>`).

Em uma organização clássica, a “`master`” é onde fica o código pronto para produção, acionando a pipeline para implantação, enquanto a `branch development` passa por alterações para em outro momento entrar em produção. É comum querer trocar entre essas branches. O `git branch` lista todas as branches locais ou cria uma nova se especificado.

`git checkout <branch>` → Muda para uma branch específica.

`git checkout -b <nome>` → Cria e muda para uma nova branch com o nome especificado.

`git merge <branch>` → Mescla a branch especificada com a branch atual.

`git pull` → Atualiza o repositório local com as mudanças do repositório remoto.

`git push` → Envia as mudanças do repositório local para o repositório remoto.

`git remote add <nome> <url>` → Adiciona um repositório remoto com um nome personalizado.

git fetch → Baixa as atualizações do repositório remoto, sem mesclar automaticamente.

git reset <arquivo> → Remove o arquivo da área de staging, mantendo as mudanças no diretório de trabalho.

git revert <commit> → Reverte um commit específico, criando um novo commit de reversão.

Um erro clássico ocorre quando comandos são executados apenas no repositório local, mas não no repositório remoto. Isso resulta em alterações criadas localmente, que depois precisam ser enviadas ao repositório remoto. É importante ter cuidado com essa prática, pois trata-se de um equívoco comum.

git stash → Armazena mudanças temporariamente, permitindo limpar o diretório de trabalho sem commit.

git stash pop → Aplica as mudanças guardadas no stash e as remove do stash.

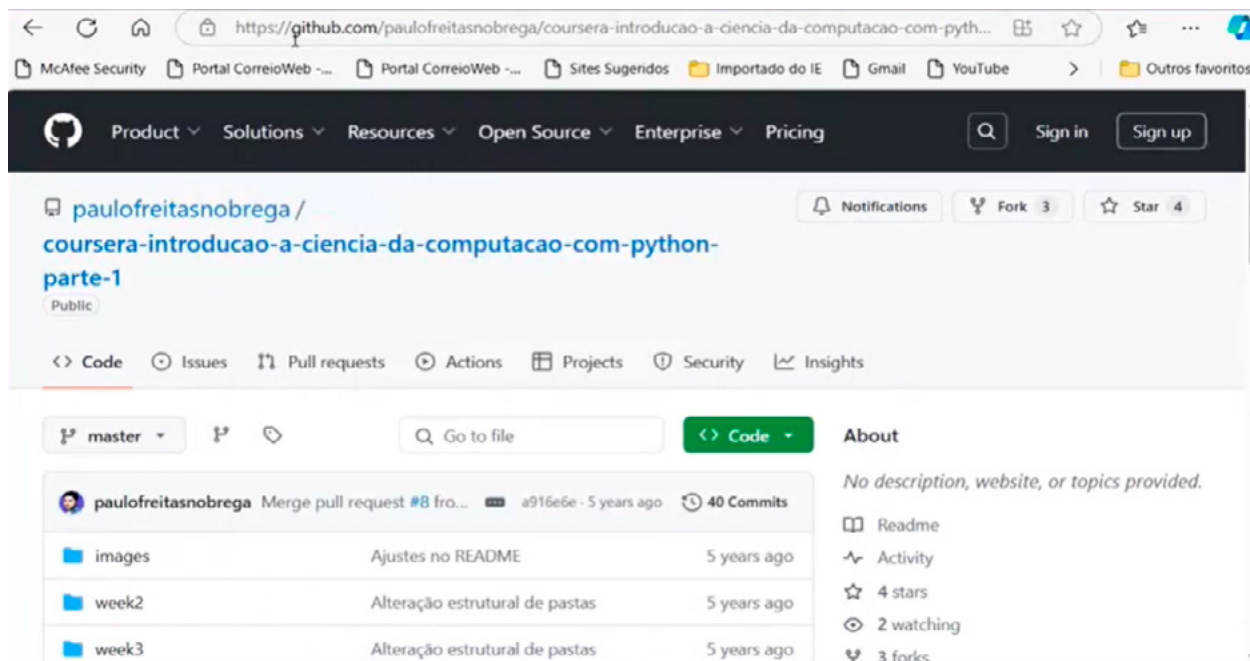
git tag <nome> → Cria uma tag (marcação) no commit atual.



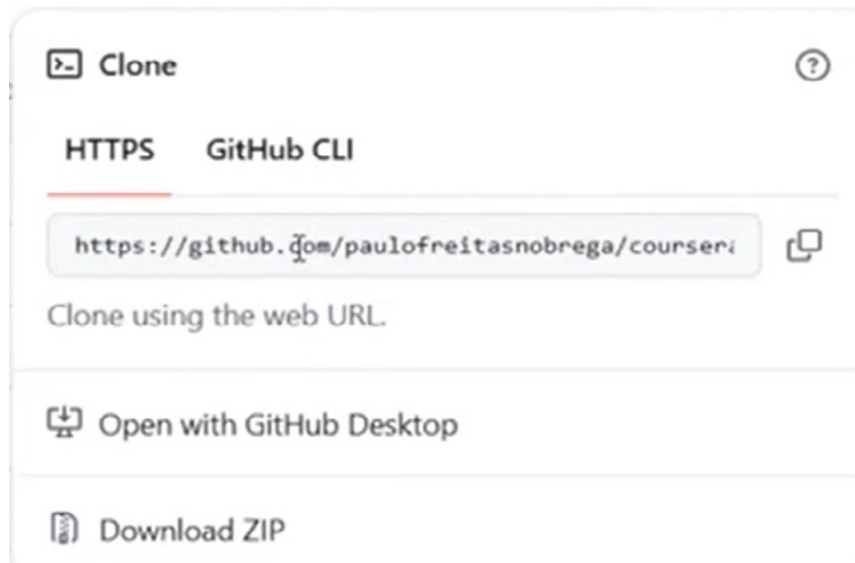
O PULO DO GATO

Embora existam interfaces gráficas para esses comandos, o conhecimento da linha de comando é exigido em provas, não havendo a opção de interagir por meio de cliques.

Será clonado o repositório do seguinte curso de python de um professor da USP:



Endereço do repositório:



Fazendo um git clone:

```

MINGW64/c:/Users/washi/OneDrive/Documents/repositorio
washi@JEDINOTEBOOK MINGW64 ~/OneDrive/Documents/repositorio
$ gh repo clone paulofreitasnobre/coursera-introducao-a-ciencia-da-computacao-com-python-parte-1
bash: gh: command not found

washi@JEDINOTEBOOK MINGW64 ~/OneDrive/Documents/repositorio
$ ghrepo clone paulofreitasnobre/coursera-introducao-a-ciencia-da-computacao-com-python-parte-1
bash: ghrepo: command not found

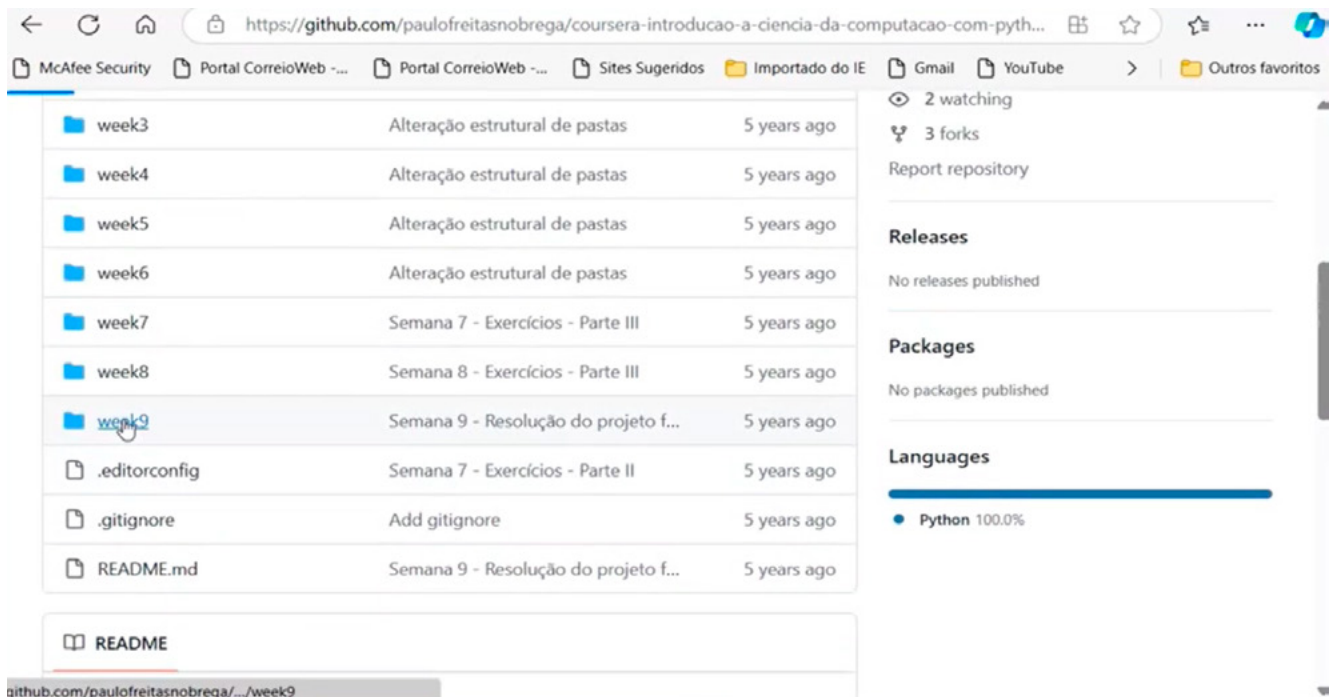
washi@JEDINOTEBOOK MINGW64 ~/OneDrive/Documents/repositorio
$ git clone A[200-https://github.com/paulofreitasnobre/coursera-introducao-a-ciencia-da-computacao-com-python-parte-1.git~
Cloning into 'coursera-introducao-a-ciencia-da-computacao-com-python-parte-1.git'...
fatal: protocol '[200-https]' is not supported

washi@JEDINOTEBOOK MINGW64 ~/OneDrive/Documents/repositorio
$ git clone https://github.com/paulofreitasnobre/coursera-introducao-a-ciencia-da-computacao-com-python-parte-1.git
Cloning into 'coursera-introducao-a-ciencia-da-computacao-com-python-parte-1'...
remote: Enumerating objects: 200, done.
remote: Total 200 (delta 0), reused 0 (delta 0), pack-reused 200 (from 1)
Receiving objects: 100% (200/200), 60.32 KiB | 753.00 KiB/s, done.
Resolving deltas: 100% (62/62), done.

washi@JEDINOTEBOOK MINGW64 ~/OneDrive/Documents/repositorio
$

```

Com isso, o repositório será clonado.



20m

Configurando email e nome para poder fazer comentários:



Device verification



Email

We just sent your authentication code via email to w*****@gmail.com. The code will expire at 10:08AM -03.

Device Verification Code

Verify

```

MINGW64:/c/Users/washi/OneDrive/Documents/repositorio/python
__pycache__/  exer01.py  exerc04.py  exercicios/  test.py
bool.py      exerc02.py  exerc05.py  exerciciovar.py

washi@JEDINOTEBOOK MINGW64 ~/OneDrive/Documents/repositorio/python (master)
$ git add .

washi@JEDINOTEBOOK MINGW64 ~/OneDrive/Documents/repositorio/python (master)
$ git commit -m "Comentário: Washington"
Author identity unknown

*** Please tell me who you are.

Run

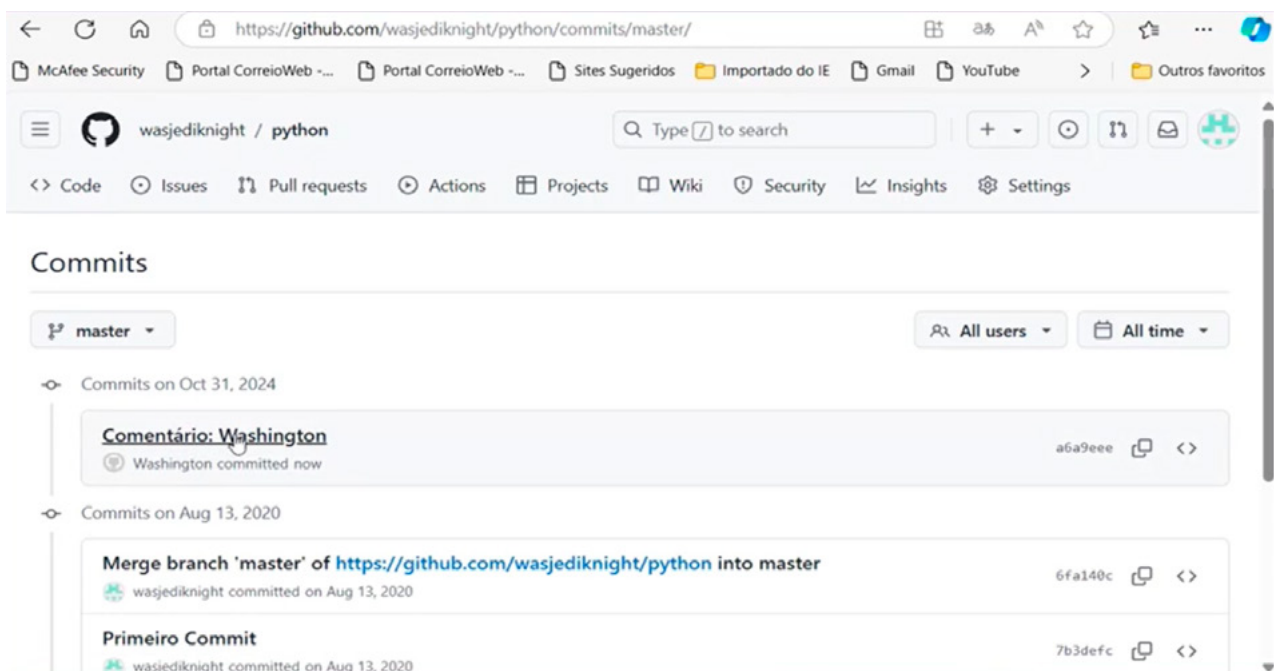
  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: unable to auto-detect email address (got 'washi@JEDINOTEBOOK.(none)')
washi@JEDINOTEBOOK MINGW64 ~/OneDrive/Documents/repositorio/python (master)
$ |

```

Agora será possível fazer comentários:



**DIRETO DO CONCURSO**

01. (INSTITUTO AOCP/2018/PRODEB/ANALISTA DE TIC I/WEB MOBILE DESIGNER) O controle de versões pode ser considerado como uma forma correta e sistemática para realizar o gerenciamento de modificações no desenvolvimento de qualquer tipo de documento, tudo isso sendo realizado de forma eficiente, extremamente organizada e prática.

Analise as seguintes informações sobre o controlador de versões Git e assinale a alternativa que aponta as corretas.

I – O Git é um software para controle de versões criado por Linus Torvalds em 2005.

II – O Git só pode ser utilizado em distribuições Linux.

III – O Git surgiu quando os desenvolvedores do kernel Linux optaram por não utilizar mais o software proprietário BitKeeper.

IV – O controlador de versões Git só deve ser utilizado em grandes projetos de desenvolvimento, pois seu uso em pequenas aplicações só acarreta um desperdício de tempo e recursos.

a) Apenas I, II e III.

b) Apenas I e III.

c) Apenas II e IV.

d) Apenas II, III e IV.

e) Apenas III e IV.



I – Antes disso, o versionador era centralizado.

II – O Git não só pode ser utilizado em distribuições Linux.

IV – O controlador de versões Git não só deve ser utilizado em grandes projetos de desenvolvimento.

O movimento do software livre surgiu como uma alternativa ao software pago. A comunidade começou a pensar em uma forma de oferecer software sem custo, onde haveria colaboração e suporte da comunidade. Esse modelo permitiu que mais pessoas tivessem acesso ao software, além de contribuir para o desenvolvimento coletivo. Essa abordagem não se limita a grandes projetos; pode ser aplicada também em projetos menores.

Praticar com a ferramenta é essencial para entender seu funcionamento, pois, na prática, erros podem surgir, e é importante saber como resolvê-los. Conhecer a ferramenta, além da teoria, ajuda na resolução de questões, principalmente quando se trata de múltipla escolha, em que o conhecimento da ferramenta facilita a eliminação de opções incorretas. A prática é fundamental para consolidar esse conhecimento.

GABARITO

01. b

Este material foi elaborado pela equipe pedagógica do Gran Concursos, de acordo com a aula preparada e ministrada pelo professor Washington Henrique Almeida.

A presente gravação tem como objetivo auxiliar no acompanhamento e na revisão do conteúdo ministrado na videoaula. Não recomendamos a substituição do estudo em vídeo pela leitura exclusiva deste material.
